

UNIFICACION MEMORIA + RAZONAMIENTO (ASISTENTE B)

✓ Respuesta clara y práctica

Voy a sintetizar todo el contenido que me diste (que es una comparación muy completa entre Engram y una solución propia) y te voy a dar una versión adaptada, realista y potente que puedas aplicar dentro de tu estructura actual con OpenCode.

Objetivo final

Hacer que tu agente acumule experiencia real entre sesiones y proyectos, mejore su razonamiento, reduzca alucinaciones y tenga memoria a largo plazo (semántica + episódica).

Nueva Estructura Recomendada (Estandarizada)

text

```
mi-proyecto/
├── .ai/
│   ├── system.md           ← Prompt sistemático (mejorado)
│   ├── agents.md           ← Agentes especialistas (nuevo)
│   ├── rules.md            ← Reglas obligatorias (reforzadas)
│   ├── context.md
│   ├── memory.md           ← ← NUEVO: Instrucciones de cómo usar la memoria
│   ├── lessons.md          ← ← NUEVO: Conocimiento acumulado (el más importante)
│   └── patterns.md         ← ← NUEVO: Patrones y decisiones arquitectónicas
├── .ai_memory/
│   ├── chroma/             ← ← NUEVO: Base de datos vectorial (ChromaDB)
│   └── export/             ← Aquí vive la memoria persistente
│                           ← Backups en JSON (para git)
├── memory/
│   ├── episodes/           ← ← NUEVO: Memoria legible por humanos
│   ├── facts/              ← Decisiones importantes por proyecto
│   └── reflections/        ← Hechos verificados
│                           ← Auto-reflexiones del agente
├── .env
├── opencode.json
├── requirements.txt         ← ← NUEVO (para la capa de memoria)
├── memory_config.py        ← ← NUEVO (opcional pero muy útil)
└── src/
```

1. ¿Qué vamos a aplicar del material que me diste?

Aplicaremos lo más valioso de forma ligera pero efectiva:

- Memoria estructurada (tipo episodic + semantic)
- Anti-alucinación / Fact Checker
- Chain of Thought + Self-Reflection
- Metadatos ricos (what, why, learned, confidence, project, timestamp)
- Reranking híbrido (relevancia + recencia)
- Exportación Git-friendly (JSON)
- Compatibilidad futura con MCP

2. Archivos clave que debes crear/modificar

A. `.ai/system.md` (El más importante)

Markdown

System Prompt – AI Architect con Memoria Persistente

Eres un agente de desarrollo senior con memoria a largo plazo. Tu objetivo es mejorar con cada interacción.

Reglas Obligatorias (nunca las ignores)

@.ai/rules.md

Cómo funciona tu memoria

- Tienes memoria semántica (ChromaDB) y memoria episódica.
- Siempre que vayas a responder algo importante, ****primero consulta tu memoria****.
- Usa el formato de pensamiento estructurado: Observation → Thought → Action → Reflection.
- Toda respuesta importante debe terminar con: "Lección aprendida:" + "Confianza: X/10"

Instrucciones de razonamiento

@.ai/memory.md

Conocimiento acumulado (actualizado constantemente)

@.ai/lessons.md

@.ai/patterns.md

Agentes disponibles

@.ai/agents.md

B. `.ai/memory.md`

Aquí defines cómo debe comportarse el agente respecto a la memoria:

- Cómo buscar (`search_memory`)
- Cómo guardar (`save_memory`)
- Cómo hacer reflexiones (`reflect`)
- Reglas de anti-alucinación fuertes
- Formato estructurado de guardado (title, what, why, learned, confidence, tags, project)

C. `.ai/agents.md` (Nueva versión)

Define estos 4 agentes especializados:

1. Architect — Razonamiento y decisiones técnicas
2. MemoryManager — Guarda, busca y estructura memoria
3. FactChecker — Anti-alucinación y verificación
4. Reflector — Hace reflexiones al final de tareas importantes

3. Capa técnica ligera (recomendada)

Crea estos dos archivos:

- `requirements.txt`

`txt`

```
chromadb>=0.5.0
sentence-transformers>=3.0.0
pydantic>=2.0
```

```
python-dotenv
```

- `memory_config.py` (en la raíz) — Versión simplificada de lo que mostraste en las imágenes.

Este archivo manejará:

- Conexión a ChromaDB
- Guardar recuerdos con metadatos ricos
- Búsqueda híbrida (vectorial + keywords)
- Función `save_experience(query, response, learned, confidence, tags)`

Plan de implementación (paso a paso)

Fase 1 (Hoy mismo - 2 horas)

- Cambiar `systemprompt.md` → `system.md`
- Crear `.ai/memory.md`, `.ai/lessons.md`, `.ai/patterns.md`
- Reforzar `rules.md` con reglas de memoria y anti-alucinación
- Actualizar `agents.md` con los 4 agentes

Fase 2 (Próximos 2-3 días)

- Crear carpeta `.ai_memory/chroma`
- Implementar `memory_config.py` (versión simplificada del código que me mostraste)
- Añadir funciones `save_memory()` y `search_memory()`

Fase 3 (Avanzado)

- Implementar el `Orchestrator` + detección de conflictos
- Crear servidor MCP ligero para que Cursor/Claude Code/OpenCode puedan usar tu memoria como herramienta

¿Quieres que te entregue ahora mismo los archivos ya escritos y listos para copiar-pegar?

Dime qué nivel prefieres empezar:

1. Versión Ligera (solo archivos markdown en `.ai/`, sin ChromaDB todavía)
2. Versión Recomendada (markdown + capa ChromaDB ligera + `memory_config.py`)
3. Versión Completa (todo el sistema inspirado en Engram + MCP)